Propuesta de Taller .NET: Sistema de Gestión de Parques solares

Taller de Programación .NET

Propuesta de Taller .NET: Sistema de Gestión de Parques solares **1**
Taller de Programación .NET 1
1. Introducción 1
2. Objetivo del Proyecto 2
3. Fundamentación de la Propuesta 2
4. Principales Requisitos Funcionales 2
5. Alcance. 4
¿Cómo funciona la captura de información? 4
6. Requerimientos no funcionales 5
7. Decisiones de diseño/tecnología de cada equipo 5
8. Cronograma 5
9. Rúbrica de evaluación. 6

1. Introducción

Uruguay se ha consolidado como un referente en la generación de corriente eléctrica utilizando energías renovables. La evolución de su matriz energética es referente a nivel regional e internacional.

Recientemente se ha anunciado la instalación de nuevos parques solares que se suman a los ya existentes parques eólicos. Puede consultar más información en la página de la Administración del Mercado Eléctrico.

Parte importante del funcionamiento de estos parques solares está soportada por un software de gestión que permite operar y monitorear su uso. En este taller vamos a construir una prueba de concepto que permita profundizar en las tecnologías de desarrollo relacionadas.



2. Objetivo del Proyecto

El objetivo principal del taller es **analizar**, **diseñar** e **implementar** una aplicación web bajo la plataforma .NET, denominada **SolarUY**, para la gestión integral de parques solares del país.. Además:

1. ganar experiencia en la plataforma .NET
2. ganar experiencia en los proceso de construcción Software (imaginar solución, priorizar, implementar)
3. ganar experiencia en una metodología profesional de desarrollo con IA

3. Fundamentación de la Propuesta

Esta propuesta se caracteriza por su enfoque general y abierto, simulando las condiciones iniciales de un proyecto de software en el ámbito profesional.

A diferencia del taller Java, donde el desarrollo de la solución estaba predeterminado (dividido en instancias concretas), en esta oportunidad, se presenta un listado inicial de necesidades surgidas de una hipotética entrevista preliminar con el cliente.

Cada equipo de desarrollo deberá asumir la responsabilidad de:

- Realizar un análisis y priorización de los requerimientos (visualizar la solución completa).
- Definir una planificación iterativa e incremental.
- Tomar decisiones técnicas y arquitectónicas fundamentadas.
- Entregar una versión funcional (Prueba de Concepto) que demuestre la viabilidad del proyecto.

Este enfoque fomenta la autonomía, la gestión de proyecto y la capacidad de adaptación, habilidades críticas en el desarrollo de software real.



4. Principales Requisitos Funcionales

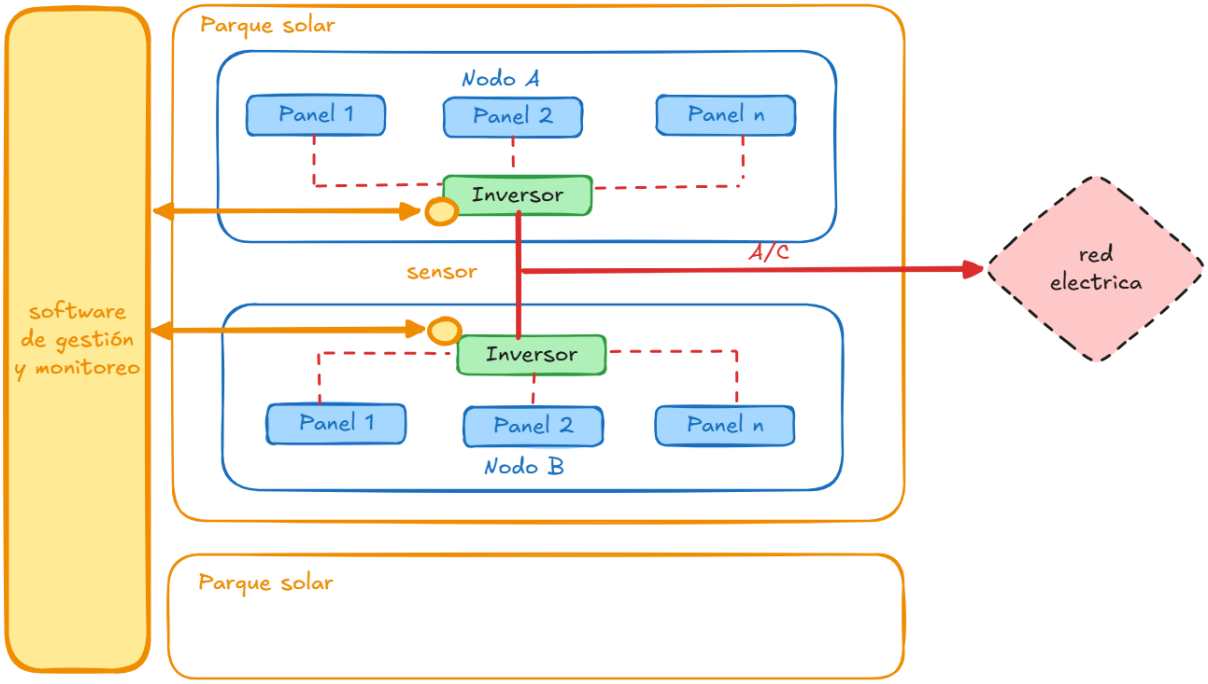
La siguiente es una lista de requerimientos a ser tomados en cuenta. Es un punto de partida para orientar la solución final.

El sistema deberá contemplar, como punto de partida, las siguientes funcionalidades:

- **Gestión multiparque**:
    - Capacidad para crear y administrar diferentes parques solares con configuraciones independientes.
    - Configurar nodos: Cada parque solar cuenta con 1 o más nodos. Un nodo es un conjunto de paneles fotovoltaicos. Los nodos tienen características que lo diferencian, por ejemplo su ubicación dentro del parque, la cantidad de paneles, etc. Cada nodo cuenta con un Inversor que convierte corriente continua (CC) en corriente alterna (CA).
    - Administrar paneles: interesa conocer los paneles con los que se cuenta, las características del mismo, en qué nodo está instalado.
    - Solo usuarios con rol de administrador podrán realizar estas operaciones.
- **Gestionar las tareas de mantenimiento de los paneles.** Éstos requieren un mantenimiento contínuo, por ejemplo limpieza, que tiene que ser realizado por personal especializado. Gestionar estas tareas es importante para el ininterrumpido funcionamiento del parque.
- **Monitoreo en tiempo real** de generación de corriente y estado de los paneles. Se quiere saber en todo momento cuánta energía está generando el parque, cada nodo y cada panel conectado. Además interesa chequear el estado de salud de cada panel para reaccionar rápidamente ante la existencia de un fallo.
- Obtener **información histórica detallada** del funcionamiento del parque: por ejemplo, generación de energía en un día, mes, año determinado. La información a registrar debe contemplar futuros requerimientos o necesidades de información.

5. Alcance.

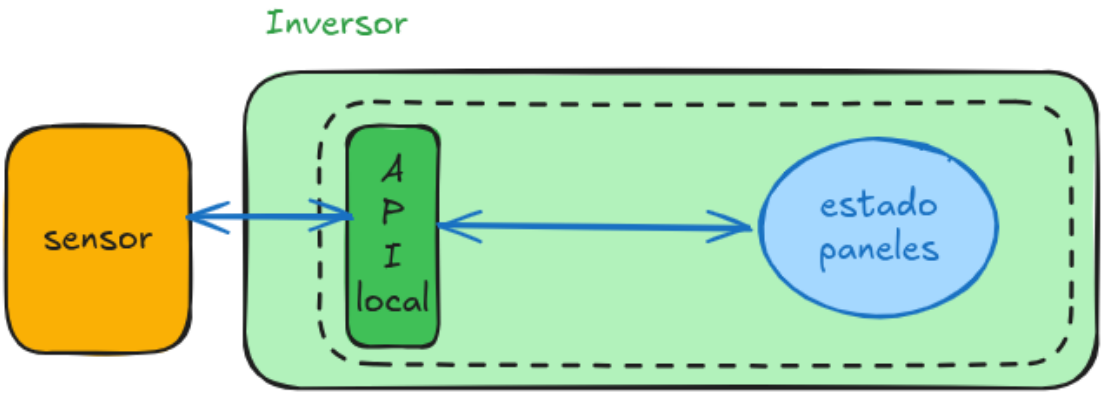
¿Cómo funciona la captura de información?



Como se observa en el diagrama, cada nodo del parque cuenta con su propio inversor que como se mencionó anteriormente, tiene como principal objetivo realizar la conversión de corriente continua a corriente alterna, necesaria para el envío de la energía a la red eléctrica.

Por su rol concentrador, el Inversor tiene información de todos los paneles solares del nodo. Además, con la ayuda de un componente acoplado llamado **sensor,** es posible interactuar con el inversor para pedirle información o mandarla a ejecutar órdenes (por ejemplo, activar/desactivar un panel, activar o desactivar el volcado de energía a la red).

El sensor es un hardware compatible con la arquitectura ARM.





En esta instancia del proyecto **no** vamos a interactuar directamente con la API local del sino que vamos a sustituir la misma por un mock o simulador del Inversor.

6. Requerimientos no funcionales

Alta disponibilidad. Se desea tener alta disponibilidad de la aplicación de gestión. No es tolerable que el sistema no esté disponible por ejemplo al momento de instalar una nueva versión de la aplicación.

Sistema de **autenticación/autorización** externo que cumpla con el estándar **OpenID**.

Posibilidad de interactuar con el sistema con **lenguaje natural** solo para las funcionalidades de consulta de información. De esta manera si un usuario quiere conocer el estado del sistema lo puede hacer con una consulta del estilo: “¿cómo está funcionando todo hoy?”

7. Decisiones de diseño/tecnología de cada equipo

Cada equipo, en base a sus preferencias e intereses deberá elegir:

- qué tecnología web utilizar:
  - ASP.NET Core Blazor
  - ASP.NET Core MVC Web App

- si deployar la aplicación en la nube
- que arquitectura utilizar:
  - monolítica
  - modular
  - microservicios con Aspire (no lo vimos en el curso pero es una buena oportunidad para investigar.)

8. Cronograma

Se seguirá una metodología ágil, con entregas parciales que permitan ajustar el desarrollo en función del feedback recibido. El calendario de hitos principales se detalla a continuación:

Semana	Día	Tarea
9	30/09/2026	Entrega de letra
	01/10/2026	
10	7/10/2026	Monitoreo
	8/10/2026	
11	14/10/2026	Primer entrega
	15/10/2026	
12	21/10/2026	Monitoreo
	22/10/2026	
13	28/10/2026	Segunda entrega
	29/10/2026	
14	4/11/2026	Monitoreo
	5/11/2026	
15	11/11/2026	Tercera entrega
	12/11/2026	
16	18/11/2026	Defensa
	19/11/2026	

En las instancias de monitoreo se discute sobre el avance actual y sobre las decisiones tomadas. En la instancia de entregas, se muestra software construido.

9. Rúbrica de evaluación.

	Excelente	Destacado	Aceptable	Deficiente
Proceso	Mantiene constancia de trabajo durante todo el taller. Presenta avances significativos en cada iteración. Participan todos los integrantes del equipo.		El proceso de trabajo es irregular o reactivo. • Los avances son limitados o no continuos, lo cual impacta el desarrollo del proyecto. • Se evidencia una metodología de trabajo que necesita mejora para ser efectiva	No participa en los monitoreos. No cumple con los plazos de entrega de las iteraciones.
Metodología de desarrollo	Usa y mejora el flujo de trabajo propuesto. Gana experiencia efectiva en el desarrollo con IA. Logra explicar cada aspecto de la aplicación desarrollada. El código generado cumple con los lineamientos de calidad			Realiza Vibe coding, no logra explicar el código generado. El código generado no cumple los estándares de calidad.
Toma decisiones	Las decisiones sobre requerimientos, diseño e implementación son oportunas, fundamentadas y consensuadas en el equipo.		La demora o dificultad en tomar decisiones, afecta el avance del taller. Las decisiones no son consensudas por el equipo.	No tiene capacidad de tomar decisiones por su cuenta, lo que impide avanzar en el taller.
Requerimientos	El sistema cumple con **todos** los requerimientos funcionales y no funcionales.		El sistema cumple con los requerimientos mínimos	El sistema no cumple con los requerimientos mínimos.
Documentación	Documentación clara y efectiva. Uso balanceado de texto e imágenes. Se focaliza en lo principal. Realiza diagramas técnicos.		No presenta la información en orden de abstracción. Mezcla detalles con información importante. No incluye diagramas técnicos	No presenta documentación o la misma no es clara o completa.